and so we visit the former first (although, as you can see, the opposite choice would be the best – but the algorithm can't know yet). Normally the distance is computed from a node's bounding envelope, but since Z intersects both S_1 and S_2, it chooses first the one whose centroid is closer. The algorithm goes in depth as much as possible, traversing a path to a leaf before back-tracking. Here we show the path to the first leaf: once there, we can update the query region, that now is the sphere centered at Z with radius equal to the distance to R, the closest point in S_{13}, which is therefore saved as "best guess" for the nearest neighbor.

Listing 10.19 shows the code for the `nearestNeighbor` method, and figures 10.20-21 shows an example of a call to the method on our example tree; as you can see it is quite compact: we just need to traverse all branches that intersects the sphere centered at the search points and whose radius is the distance to current nearest neighbor, and update the best value found in the process.

This simplicity and cleanness are not unexpected: we have done the hard work in the design and creation of the data structure, and now we can just enjoy the benefits!

Listing 10.19 The nearestNeighbor method for Ss-tree

```
function nearestNeighbor(node, target, (nnDist, nn)=(inf,
null))              #1
  if node.leaf then
                                                        #2
```